

START HERE · NO OAUTH REQUIRED

PKCE, from the Beginning

One attack, one fix, explained slowly. What an authorization code is, how it gets stolen, and the five lines of code that make stealing it useless.



PROVE YOU STARTED THE FLOW YOU ARE FINISHING
RFC 7636 · REQUIRED FOR EVERY CLIENT IN OAUTH 2.1

For beginners. Worked examples in 7 languages, and a lab you can break on purpose.

WRITTEN BY

Smiroh Dev

Smiroh Publishers
COMPANION VOLUME

PKCE, from the Beginning

Smiroh Dev

PUBLISHER	Smiroh Publishers
EDITION	First edition
PUBLISHED	September 2026
FORMAT	Electronic book (PDF), 36 pages
ISBN (EB00K)	To be assigned
SUBJECT	Computers / Security / Web Development
COMPANION TO	The Ultimate Developer's Guide to OAuth 2.1 and PKCE

Copyright © 2026 Smiroh Dev. All rights reserved.

Published by Smiroh Publishers. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

WHO THIS BOOK IS FOR

Developers who have been told to “add PKCE” and would like to understand what they are adding and why. No prior knowledge of OAuth is assumed. If you already know the authorization code flow and want depth on tokens, scopes, rotation and OpenID Connect, the companion volume is the better book — this one stays on a single subject and takes its time.

STANDARDS REFERENCED

PKCE is defined by RFC 7636. Its role in current practice is set by the OAuth 2.1 draft and by RFC 9700, *Best Current Practice for OAuth 2.0 Security*. Those documents are normative; where this book and a specification disagree, the specification governs.

TRADEMARKS

All trademarks and registered trademarks appearing in this book are the property of their respective owners. They are used editorially, with no intention of infringement.

LIMIT OF LIABILITY AND DISCLAIMER OF WARRANTY

The publisher and the author make no representations or warranties with respect to the accuracy or completeness of the contents of this work and specifically disclaim all warranties. The advice and strategies contained herein may not be suitable for every situation. Neither the publisher nor the author shall be liable for damages arising herefrom. The code in this book is illustrative: it is written for clarity, not for production, and it has not been audited.

SECURITY NOTICE

The attack described in Chapter 3 is presented so that you can defend against it. Exercising it against systems you do not own or have permission to test is, in most jurisdictions, a criminal offense.

What is in this book

Why PKCE exists, how it works, typing it out yourself, and what people get wrong.

PART I · WHY PKCE EXISTS

01	The handoff	What OAuth is doing, in plain words	4
02	The authorization code	A one-time ticket, and the two channels it travels	6
03	How the code gets stolen	The interception attack, step by step	7
04	Why a client secret will not save you	Public clients, and what “secret” means in an app anyone can download	9

PART II · HOW PKCE WORKS

05	The idea	Prove you are the app that started the flow	10
06	The three parameters	code_verifier, code_challenge, code_challenge_method	11
07	A complete run, with real values	Every request and response, nothing elided	13
08	Why a hash, and what base64url is	One-way functions for people who have not met one	15
09	The attack, defeated	The same theft, now worthless	18
10	plain, S256, and the downgrade	Why one of the two methods is a trap	19

PART III · DO IT YOURSELF

11	Generate the pair	Shell, JavaScript, Python, Java, Kotlin, Swift, C#	21
12	Build the authorization request	Every parameter, and what it is for	24
13	Exchange the code	The one request that matters	25
14	Break it on purpose	Four failures, and the exact errors they produce	26
15	Run it against the simulator	The lab in this repository, end to end	28

PART IV · GETTING IT RIGHT

16	PKCE, state and nonce	Three parameters, three different jobs	29
17	The mistakes beginners make	Symptoms, causes, fixes	30
18	What PKCE does not do	The gaps it was never meant to close	32
19	The checklist	What to verify before you ship	33
–	Quick reference	One page to keep open while you work	34
–	Glossary	Every term this book uses	35
–	Questions people ask	And where to read next	36

The handoff

Before PKCE can make sense, one thing has to: what is actually happening when an app says “Sign in with ...” and a different website appears.

You install a photo-printing app. It wants the photos in your cloud drive. There are two ways it could get them, and only one of them is sane.

The first way is to ask for your cloud password and log in as you. This is a disaster for reasons that are easy to list once you start: the app now has access to everything, not just photos; the only way to take that access back is to change your password, which breaks every other app at the same time; the cloud provider cannot tell the app's requests from yours, so nothing can be audited; and your two-factor authentication has nowhere to fit.

The second way is the one the whole industry settled on. The app never sees your password. Instead you are sent to the cloud provider, you log in *there*, you are asked whether this app may read your photos, and if you agree the provider hands the app a **token** — a narrow, expiring credential that permits exactly that and nothing else. That arrangement is called **delegated authorization**, and OAuth is the protocol that performs it.

The valet key. Some cars come with a second key that opens the door and starts the engine but will not open the boot and limits the car to 30 miles an hour. You hand it to someone you trust a little, for one job, and you can have it back. A token is a valet key. Your password is the key to the house.

The four parties

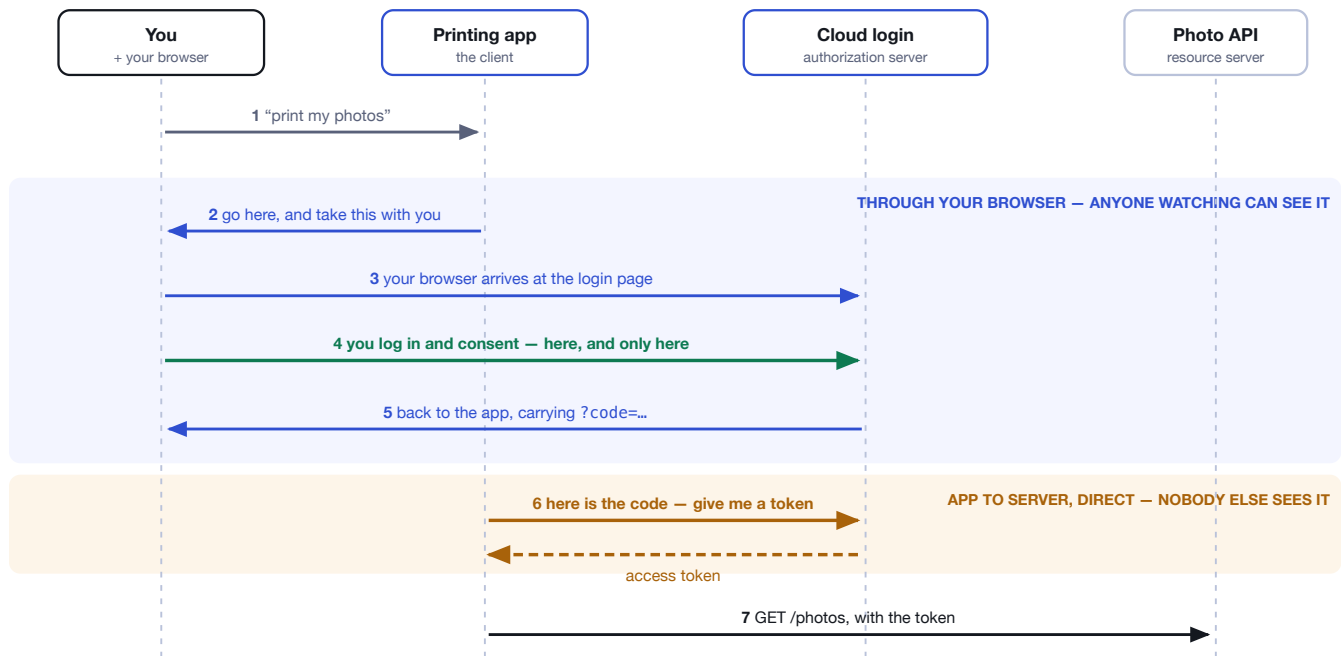
Every OAuth conversation has the same cast. Learning the names now makes the rest of this book shorter.

NAME	IN OUR EXAMPLE	WHAT IT DOES
Resource owner	You	Owns the data and is the only one who can agree to share it.
Client	The printing app	Wants the data. Confusingly, “client” means the <i>application</i> , not you.
Authorization server	Your cloud provider's login	Authenticates you, asks your consent, and issues tokens.
Resource server	The cloud photo API	Holds the photos and accepts the token as permission to serve them.

Two of those often live at the same company but they are separate jobs, and keeping them separate in your head matters later: the authorization server is the only party that ever sees your password, and the resource server is the only party that ever sees your photos.

What the handoff looks like

The app cannot simply ask the authorization server for a token, because the authorization server has no idea who you are until you have logged in — and you must log in somewhere the app cannot watch you type. So the app steps aside. It sends your browser to the authorization server, and waits.



Your password travels on arrow 4 only. The app never sees it, and never needs to.

The handoff, in order. Time runs downwards. Steps 2 to 5 happen *through your browser*, which means they appear in URLs, in history, and in anything watching the device. Step 6 is a direct call from the app to the server that nothing else takes part in. That difference is the whole of this book.

Notice what arrives at step 5. Not a token — a **code**. The app is not trusted with a token just because a browser turned up carrying something. It has to perform one more step, in private, to trade that code for the real credential. The next chapter looks at why the protocol is built that way.

One word is worth being precise about. When we say the app “redirects your browser”, nothing clever is happening: the app returns an ordinary HTTP redirect, or opens a URL, and the browser goes there. Everything the app wants the authorization server to know is simply part of that URL — and everything on arrows 2 to 5 is therefore a URL that you, your browser, your operating system, and any software installed alongside them can read.

The authorization code

A short string that is worth a token. Understanding why the protocol bothers with it is the last thing you need before the attack makes sense.

At step 5 the browser came back to the app carrying something like this:

```
// the browser lands back on the app's redirect URI
com.printingapp://callback?code=SpLx10BeZQQYbYS6WxSbIA&state=af0ifjsldkj
```

That `code` is the authorization code. It has four properties worth committing to memory, because every one of them exists for a reason:

Short-lived	Typically valid for 30 to 60 seconds. It is meant to be used immediately.
Single use	The server marks it spent the first time it is redeemed. A second attempt fails — and a good server also revokes any token it already issued for that code, on the assumption that two parties are fighting over it.
Bound to one client	It was issued for a particular <code>client_id</code> and redirect URI.
Not a credential	On its own it opens nothing. It has to be exchanged.

Why not just send the token?

An older version of OAuth did exactly that: the authorization server put the access token straight into the redirect URL. It was called the *implicit flow*, and OAuth 2.1 removes it. The reason is the one thing you now know about arrows 2 to 5: they are URLs.

A URL is written into browser history. It is sent in the `Referer` header to whatever the page loads next. It is logged by proxies, by the browser's own diagnostics, and by any extension with permission to read tabs. Putting a token that is valid for an hour into a URL means putting it into all of those places.

The authorization code exists so that the *valuable* thing — the token — never travels that way. The code goes through the browser; the token comes back over a direct, private connection from the app to the server. Even if the code leaks into a log, it expires in under a minute and can only be spent once.

This is good design. The code is deliberately a low-value, fast-expiring bearer of a one-time right, precisely so that the channel that cannot be kept private carries as little value as possible.

So where is the problem?

Everything above is sound — as long as one assumption holds. The server hands the token to whoever turns up with a valid code, so the whole design rests on the code reaching *only* the app that started the flow.

On a web server that assumption is reasonable. On a phone, or in a single-page app, it turns out not to be. That is the subject of the next chapter.

How the code gets stolen

The authorization code has to be delivered back to the app somehow. On a phone, “somehow” turns out to be a door that more than one app can stand behind.

A website's redirect URI is a URL: `https://app.example.com/callback`. The browser resolves that name through DNS and TLS, so it is genuinely hard for another party to receive it.

A mobile app has no domain name. For years the way an app said “send the browser back to me” was to register a **custom URL scheme** with the operating system:

```
// the printing app claims this scheme at install time
com.printingapp://callback
```

When the browser is told to go to a `com.printingapp://` address, the operating system looks up which installed app claimed that scheme and hands it over. Here is the problem, and it is embarrassingly simple: **on classic custom schemes, nothing stops a second app from claiming the same one.** There is no registry, no proof of ownership, and no check. An attacker publishes a free wallpaper app that quietly declares `com.printingapp` in its manifest, and waits.

What happens when two apps claim one scheme is platform-specific and, from a security point of view, all of the answers are bad: the system may pick the one installed first, or last, or show the user a chooser they will not understand. The attacker does not need to win every time. Winning sometimes is enough.

This is not theoretical. Authorization code interception through custom-scheme collision is the attack RFC 7636 was written to stop. Its abstract says so in the first sentence. Every mitigation in this book exists because this worked.

Other doors into the same room

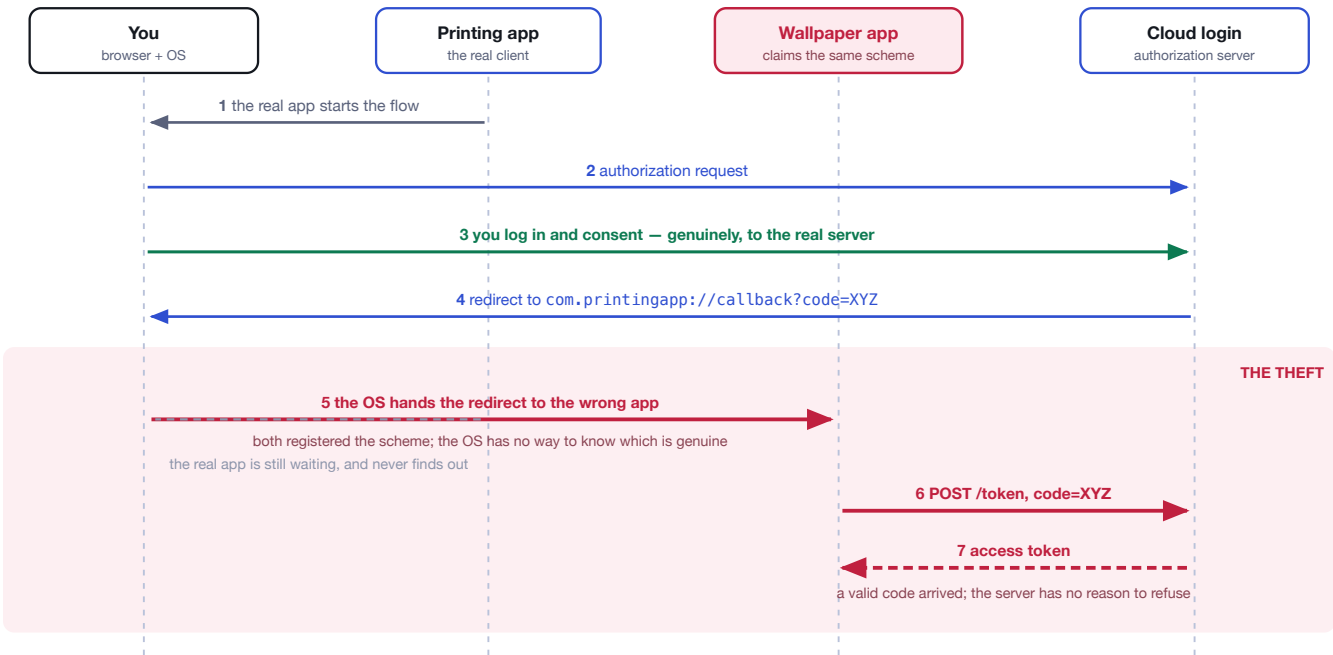
The custom-scheme collision is the classic case, but the code is exposed anywhere the redirect is visible:

- A malicious or compromised **browser extension** that can read tab URLs.
- An **open redirector** on the client's own domain, which lets an attacker bounce the code to a site they control.
- **Referer leakage**, where the callback page loads a third-party script and the browser helpfully sends the full URL along.
- **Shoulder-level logging**: proxies, crash reporters, analytics tools and server access logs that record full URLs by default.
- On a shared or infected device, anything reading **browser history**.

Notice that none of these require breaking TLS or guessing a secret. They require only being somewhere the URL passes.

The theft, step by step

Follow the code. Everything up to step 4 is the normal, correct flow — which is exactly what makes this attack unpleasant. Nothing has gone wrong yet at the point where everything goes wrong.



Authorization code interception. You did nothing wrong. The authorization server did nothing wrong. The code was valid, single-use and unexpired — it was simply handed to the wrong hands, and nothing in the protocol so far lets the server tell the difference.

Step 7 is the one to sit with. The server is behaving correctly by its own rules. It received a code it issued minutes ago, unexpired and unspent, for a registered client, and it did what the specification says: it returned a token. **The server cannot distinguish the thief from the app, because the code is the only evidence it has.**

Why the operating system cannot help

It is natural to ask why the phone does not simply refuse to give the redirect to an app that does not own the scheme. The answer is that, with a classic custom scheme, it has nothing to check against.

REDIRECT MECHANISM	CAN THE PLATFORM VERIFY THE CLAIM?
https:// (web)	Yes. DNS and TLS prove who owns the domain.
Custom scheme	No. The string is declared in a manifest. Nothing anywhere states who is entitled to it.
App Links (Android) Universal Links (iOS)	Yes. The app declares a domain, and the platform fetches a file from that domain to confirm the association before honoring the link.

Verified links are the right long-term answer, and worth adopting. But they are a platform feature: they need a domain, a published association file, a recent OS, and an app installed from a store. PKCE needs none of those and does not depend on the attacker's environment being well behaved — which is why it is the mandatory part.

Why a client secret will not save you

There is an obvious objection: make the app prove who it is when it redeems the code. It is a good instinct, and for a whole class of application it is impossible.

Web applications already do this. When a server-side app exchanges a code it also sends a **client secret**, issued at registration. The thief in the last chapter has the code but not the secret, so step 6 fails. Problem solved — for a server. For anything the user installs or downloads, no, because of a distinction OAuth draws at the start.

CLIENT TYPE	DEFINITION, AND WHAT IT MEANS IN PRACTICE
Confidential	Can keep a secret, because the secret lives on a machine the developer controls. A server-side web app, a backend service, a daemon.
Public	Cannot keep a secret. The whole application is in the user's hands. A mobile app, a desktop app, a single-page app, a CLI tool.

“Cannot” is a strong word. Is it really?

It really is. Put a secret in a mobile app and it ships inside the binary. Anyone can download that binary from the store, and the tools to look inside it are free, standard and a single command:

```
# pull the strings out of an app bundle – no skill required
$ unzip -q PrintingApp.ipa -d x && strings x/Payload/PrintingApp.app/PrintingApp \
  | grep -iE 'secret|client_id'
client_secret=7Fd2c9Ab...      ← there it is
```

A single-page application is worse still: its “secret” is served to the browser as JavaScript, one View Source away. Obfuscation changes nothing. The secret must be present for the app to use it, so it is present for the attacker too. It is not a secret; it is a slightly inconvenient constant.

What we actually need

So the requirement is sharper than “prove who you are”. A public client cannot prove *who it is*, because any proof it carries can be copied out of it before the flow even starts. What it can do is prove something narrower:

Prove that the party redeeming this code is the same party that requested it — without shipping any long-lived secret at all.

That is a much smaller claim, and it has a neat solution: instead of using a secret the developer baked in, the app *invents a fresh secret for every single login*. Nothing is shipped, so nothing can be extracted. That idea is PKCE, and Part II is about how four lines of arithmetic turn it into a working proof.

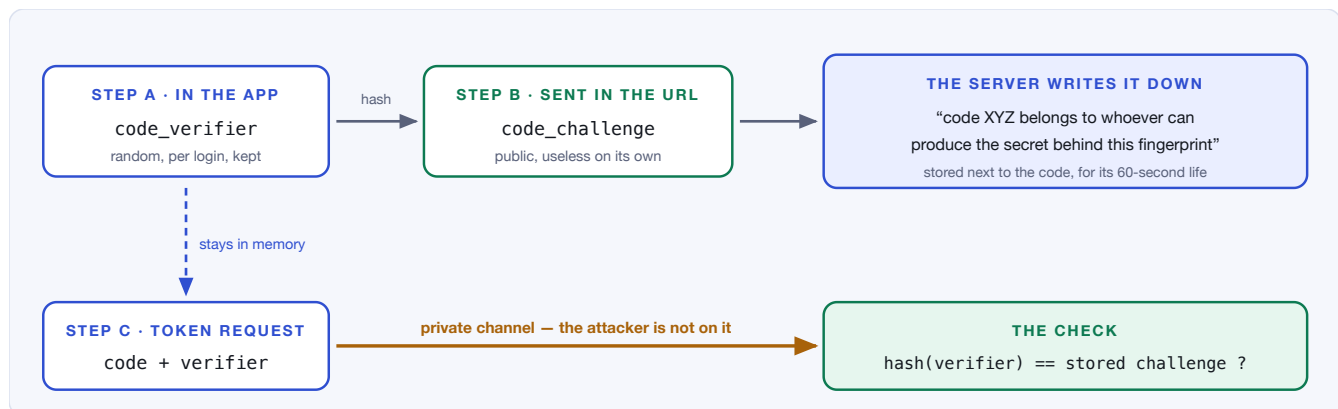
The idea

PKCE is one sentence long. Everything else — the parameter names, the hashing, the base64 — is mechanics in service of that sentence.

Before starting a login, the app invents a random secret. It sends a one-way fingerprint of that secret with the authorization request, and the secret itself with the token request. The server checks that they match.

The name is an acronym: **P**roof **K**ey for **C**ode **E**xchange. It is said “pixy”. The “proof key” is the random secret; the “code exchange” is step 6, the moment the code becomes a token. PKCE attaches a proof to that exchange.

Follow what that buys us against the thief in Chapter 3. The fingerprint went out in the authorization URL, so the attacker may well have seen it. The code came back through the redirect, so the attacker has that too. But the secret itself never went anywhere near the browser — it stayed in the app's memory the entire time. Without it, the code the attacker stole cannot be redeemed.



One secret, two halves, one comparison. The fingerprint travels where an attacker can see it. The secret travels only where they cannot. The server never needs to know which app is genuine — only that whoever is finishing the flow is whoever started it.

The three parameters

PKCE adds two parameters to the authorization request and one to the token request. That is the entire protocol change.

PARAMETER	SENT IN	WHAT IT IS
code_verifier	token request	The secret. A high-entropy random string, 43 to 128 characters, from the set <code>A–Z a–z 0–9 – . _ ~</code> . Generated fresh for every authorization attempt and never reused.
code_challenge	authorization request	The fingerprint. With <code>S256</code> , the base64url encoding of the SHA-256 hash of the verifier.
code_challenge_method	authorization request	How the challenge was derived: <code>S256</code> or <code>plain</code> . Always send <code>S256</code> . Chapter 10 explains why the other one is a trap.

The rules, precisely

Length and alphabet

The verifier must be at least 43 and at most 128 characters, drawn from the unreserved URL characters. The lower bound is not arbitrary: 43 base64url characters carry 256 bits of entropy, which is the point. The alphabet is chosen so the value survives being placed in a URL or a form body without any escaping.

43 characters, not 43 bytes. The usual mistake is to generate 43 random *bytes* and base64url-encode them, producing a 58-character verifier. That is perfectly legal and slightly stronger than required. The mistake that bites is the reverse: generating 32 bytes, encoding them to 43 characters, and then assuming the *bytes* are the verifier. The verifier is the text you produced, and it is that text you must hash and that text you must send later.

The derivation

```
// S256 – the only method you should use
code_challenge = BASE64URL-ENCODE( SHA256( ASCII( code_verifier ) ) )

// plain – permitted by the RFC, and a bad idea (chapter 10)
code_challenge = code_verifier
```

Three details hide in that one line, and each of them is a source of bugs:

- **Hash the characters, not a decoding of them.** SHA-256 runs over the ASCII bytes of the verifier string exactly as you will later transmit it.
- **The hash is 32 raw bytes.** Hash them, do not hex-encode them first. A hex string base64s to 64 characters, and a 64-character challenge is the classic symptom of this bug — the correct length is 43.
- **base64url, without padding.** That means `-` and `_` instead of `+` and `/`, and no trailing `=`. Chapter 8 shows what that means concretely.

What the server does

The server's side is three lines of logic, and it is worth knowing them because every error message you will meet comes from one of them:

1. At the authorization request: store `code_challenge` and `code_challenge_method` alongside the code it is about to issue.
2. At the token request: take the `code_verifier` the app sent, apply the stored method to it, and compare the result to the stored challenge.
3. Equal, and the code is valid and unspent? Issue the token. Otherwise refuse, and do not say why in any detail.

A complete run, with real values

Every request and response of one successful login, with nothing left out and nothing abbreviated. These values are genuine: you can reproduce the hash yourself.

Step 1 — the app invents a secret

The app generates 43 random bytes and base64url-encodes them. Count the result: it is **58** characters, not 43. That is the distinction from the last chapter — the RFC's minimum of 43 is a count of *characters*, and 43 bytes of randomness comfortably exceeds it.

```
// code_verifier – 58 characters, kept in memory, never transmitted yet
M25iVXpKU3puUjFaYWg3T1NDTDQtcW1R0UY5YXlwalNoc0hhakxifmZHag
```

Step 2 — the app derives the fingerprint

SHA-256 over the ASCII of that string, base64url-encoded, unpadded.

```
// code_challenge – 43 characters, safe to publish
qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es
```

Step 3 — the authorization request

The app opens this URL in the system browser. The two PKCE parameters are the last two.

```
GET https://login.example.com/authorize
?response_type=code
&client_id=printing-app
&redirect_uri=com.printingapp%3A%2F%2Fcallback
&scope=photos.read
&state=af0ifjsldkj
&code_challenge=qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es
&code_challenge_method=S256
```

You log in and consent. The server stores the challenge and the method against the code it is about to mint.

Step 4 — the redirect back

```
HTTP/1.1 302 Found
Location: com.printingapp://callback?code=SpLxl0BeZQQYbYS6WxSbIA&state=af0ifjsldkj
```

The app checks that `state` is the value it sent, then proceeds. Note what is *not* in this URL: the verifier. It has not left the app.

Step 5 — the token request

Now the private call, app to server over TLS. The verifier appears for the first and only time.

```
POST /token HTTP/1.1
Host: login.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=SpLx10BeZQQYbYS6WxSbIA
&redirect_uri=com.printingapp%3A%2F%2Fcallback
&client_id=printing-app
&code_verifier=M25iVXpKU3puUjFaYWg3T1NDTDQtcw1R0UY5YXlwa1Noc0hhakxifmZHag
```

Step 6 — the server checks, and answers

The server hashes what it received and compares it with what it stored in step 3.

```
// server side, in effect
BASE64URL(SHA256(verifier_received)) == stored_challenge → true
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{
  "access_token": "eyJhbGciOiJIUzI1NiIsImtpZCI6IjIwMjYtMDkifQ...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "photos.read"
}
```

And if the verifier is wrong

Change one character and the comparison fails. The answer is deliberately terse: it will not tell an attacker which part they got wrong.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "error": "invalid_grant",
  "error_description": "PKCE verification failed"
}
```

That is the entire protocol. Two extra query parameters, one extra form field, one hash comparison. The rest of this book is about doing it correctly and knowing what it does not cover.

Why a hash, and what base64url is

Two pieces of machinery that everything above depends on, explained for someone who has not met either.

A one-way function

SHA-256 takes any input and produces 32 bytes. It has one property that makes PKCE work: computing the output from the input is instant, and computing the input from the output is, as far as anyone knows, impossible.

Watch how completely the output changes when the input barely does:

```
// one character different
SHA256("hello") = 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e7304...
SHA256("help") = fdd7585e08c4e2afd71dcabdb4636c89d557a3f42db9e2040c8b...
```

There is no partial credit and no gradient to climb. An attacker holding a challenge has no better strategy than guessing verifiers and hashing each one, and the verifier has at least 256 bits of entropy. The number of guesses required is not large; it is *unreachable* — more operations than there are atoms available to perform them.

This is why the challenge is safe in a URL. Publishing the fingerprint gives an attacker nothing they can use, because the only way back to the secret is through a door that does not exist.

Why not send the secret both times?

A fair question. If the app sent the same random string in the authorization request and again in the token request, the server could compare them directly — no hashing at all. That is exactly what `plain` does, and it is broken for one reason: the authorization request travels through the browser. An attacker who can read the redirect can read the authorization URL too. They would capture the secret on its way out, and use it on the way back.

The hash exists so that the thing sent through the untrusted channel is *not* the thing needed to redeem the code.

base64url, concretely

Hashes are raw bytes, most of which are not printable characters. base64 is the standard way to write arbitrary bytes as text: it takes three bytes at a time and emits four characters from a 64-character alphabet.

Ordinary base64 uses `+` and `/` as its last two characters and pads the end with `=`. All three are hostile inside a URL: `+` means a space in a form body, `/` is a path separator, and `=` separates a parameter from its value. `base64url` is the same encoding with those three problems removed.

CHARACTER	BASE64	BASE64URL
62nd	<code>+</code>	<code>-</code>
63rd	<code>/</code>	<code>_</code>
padding	<code>=</code> (kept)	<code>=</code> (removed entirely)

```
// the same 32 bytes, both ways
base64      qjrzSW9gMiUgpUvqgEPE4/+8swvyCtf0Vvg55o5S/es=
base64url   qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es
               ^^                               ^ ← and no trailing =
```

Three bytes in, four characters out

base64 works in blocks. It takes 24 bits — three bytes — and re-cuts them into four groups of six bits, because a 6-bit number has 64 possible values, one per character of the alphabet.

```
// the three bytes 0xAA 0xB3 0xF3
bytes   10101010 00111011 11110011
regroup 101010 100011 101111 110011
value    42     35     47     51
base64    q      j      v      z
```

Since three bytes always become four characters, a 32-byte hash is not a multiple of three: 32 bytes is ten full blocks with two bytes left over. Those two bytes produce three characters, giving 43 in total. Standard base64 then adds one `=` to round the final block up to four; `base64url` drops it. That is the whole reason the number 43 keeps appearing in this book.

Getting it wrong, and how you will know

Almost every PKCE bug a beginner hits is an encoding bug, and each has a distinctive fingerprint. Learn to read the length of a challenge and you can diagnose most of them at a glance.

LENGTH	WHAT YOU PRODUCED	THE MISTAKE
43	base64url of 32 raw bytes	Correct. This is what a valid S256 challenge always looks like.
44	base64 with padding kept	You left the trailing <code>=</code> on. Strip it.
64	base64url of a 64-character hex string	You hex-encoded the hash before base64ing it. Hash to bytes, then encode the bytes.
58	the verifier itself	You sent the verifier as the challenge — that is <code>plain</code> , while claiming S256. It will fail.
86	base64url of a SHA-512 hash	Wrong algorithm. S256 means SHA-256, always.

The one that is hardest to see. If your challenge contains a `+`, a `/` or an `=`, you used standard base64. It may even work in testing, because some servers tolerate URL-encoded forms of those characters — and then fail in production behind a proxy that normalizes them differently. The character set is part of the specification, not a formatting preference.

A note on randomness

The verifier must come from a cryptographically secure random source. The distinction matters more than it sounds:

DO NOT USE	USE
<code>Math.random()</code>	<code>crypto.getRandomValues()</code>
<code>random.random()</code>	<code>secrets.token_urlsafe()</code>
<code>new Random()</code>	<code>SecureRandom</code>
<code>rand()</code>	<code>/dev/urandom</code> , <code>arc4random_buf()</code>

An ordinary random number generator is built to be fast and statistically even, not unpredictable. Given a few of its outputs, its internal state — and therefore every future output — can often be reconstructed. An attacker who can predict your next verifier does not need to steal it.

Do not derive the verifier from anything else either: not the time, not a session identifier, not a hash of the user's name. It has exactly one job, which is to be unguessable.

The attack, defeated

The same thief, the same stolen code, the same server. One field has been added to the token request, and the theft is now worth nothing.

Run Chapter 3 again with PKCE in place. Steps 1 to 5 are unchanged — the attacker still wins the race for the custom scheme and still ends up holding a perfectly valid authorization code. The difference is entirely in what happens next.

THE ATTACKER TRIES

```
POST /token
code=Sp1x10BeZQQYbYS6WxSbIA
client_id=printing-app
code_verifier=???
```

They have the code. They saw the challenge go past in the URL. Neither helps:

- X Guessing the verifier: 2^{256} attempts
- X Reversing the hash: no known method
- X Replaying the challenge: it is not the verifier, and the server hashes what it is given before comparing

Result: invalid_grant. The code dies unspent.

THE REAL APP SUCCEEDS

```
POST /token
code=- never received it -
client_id=printing-app
code_verifier=M25iVXpKU3pu...
```

In this run the real app lost the race, so it simply times out and starts again.

When it *does* receive the redirect, it is the only party in the world holding the verifier for that code — so the hash matches and the token is issued.

The user notices nothing. The attacker gets an error page and an unusable string.

Same theft, different ending. PKCE does not stop the code being stolen. It makes a stolen code useless, which is a better goal: it does not depend on the operating system, the browser, or the user behaving correctly.

Worth saying plainly. PKCE is not a lock on the door the code came through. It is the rule that the code is only redeemable by whoever knows a secret that never went through that door at all.

plain, S256, and the downgrade

RFC 7636 defines two methods. Only one of them is worth using, and the existence of the other creates an attack of its own.

What plain does

```
// plain: the challenge IS the verifier
code_challenge = code_verifier
code_challenge_method = plain
```

Now re-read Chapter 3. The attacker who can intercept the redirect can almost always also observe the authorization request — they are the same channel, often the same URL bar, the same history file, the same proxy log. With `plain`, that request contains the verifier in clear text. The attacker copies it, waits for the code, and redeems both together.

plain restores the original attack. It satisfies the letter of “we implemented PKCE” while removing the property that made PKCE work. The RFC itself says clients must use S256 if they can, and that they can unless the platform genuinely has no SHA-256 — a condition that has not meaningfully existed for years.

So why is it in the specification?

RFC 7636 was published in 2015, when it was still plausible that some constrained device had no SHA-256 available. `plain` was the concession that let such a client participate at all, on the reasoning that a weak binding is better than none: it still defeats an attacker who sees the redirect but not the original request.

That reasoning has not aged well, because the two are usually the same channel. Current guidance is unambiguous — the OAuth 2.1 draft and RFC 9700 both expect `S256`, and a server built today has no reason to offer the alternative.

IF YOU CAN SEE...	PLAIN	S256
only the redirect	SAFE	SAFE
the authorization request too	BROKEN	SAFE
and you can rewrite it	BROKEN	DEPENDS ON THE SERVER

The downgrade attack

Here is the subtler problem, and it is a *server* bug rather than a client one. Suppose an app correctly sends `S256`. An attacker intercepts the authorization request, and rewrites two parameters before it reaches the server:

```
// what the app sent
code_challenge=qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es&code_challenge_method=S256

// what the attacker rewrites it to
code_challenge=attacker-picked-value&code_challenge_method=plain
```

The server now stores the attacker's own value with the method `plain`. When the attacker later presents `code_verifier=attacker-picked-value`, it matches perfectly — and the real app's carefully generated secret was never involved.

How a server must prevent it

- **Support only S256**, and reject `plain` outright. This is what the OAuth 2.1 draft expects of a new server, and it closes the hole completely.
- If `plain` must be supported for a legacy client, **record per client whether it is permitted**, and refuse a downgrade for any client that has ever used S256.
- **Reject a token request with no code_verifier** when a challenge was stored. Silently ignoring the missing field is the same bug wearing a different hat: the attacker just omits the parameter.

How a client can tell

An authorization server publishes what it supports in its discovery document. Look for this before you trust it:

```
GET /.well-known/oauth-authorization-server

{
  "code_challenge_methods_supported": ["S256"]           ← good
  "code_challenge_methods_supported": ["plain", "S256"]  ← usable, but ask why
  // field absent entirely → the server may not support PKCE at all
}
```

Generate the pair

The whole of PKCE on the client side is these few lines. Each example produces a 58-character verifier and its 43-character challenge.

At a shell prompt

Useful for poking at a server by hand, and the quickest way to convince yourself the arithmetic is nothing special.

```
# generate a verifier, then derive its challenge
VERIFIER=$(openssl rand -base64 43 | tr '+/' '_-' | tr -d '=\\n')
CHALLENGE=$(printf %s "$VERIFIER" | openssl dgst -binary -sha256 \
    | openssl base64 | tr '+/' '_-' | tr -d '=\\n')

echo "verifier ${#VERIFIER}"      # 58
echo "challenge ${#CHALLENGE}"   # 43
```

Note `printf %s`, not `echo`. `echo` appends a newline, and hashing that newline gives a different, wrong challenge. This single byte is one of the most common sources of `invalid_grant` when testing by hand.

JavaScript — Node 16 and later

```
import { randomBytes, createHash } from "node:crypto";

const b64url = (buf) => buf.toString("base64url");

const verifier = b64url(randomBytes(43));
const challenge = b64url(createHash("sha256").update(verifier).digest());
```

JavaScript — the browser

There is no `base64url` convenience in the browser, so the conversion is explicit. `crypto.subtle` is asynchronous and is only available on a secure origin.

```
const b64url = (bytes) =>
  btoa(String.fromCharCode(...new Uint8Array(bytes)))
  .replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");

const verifier = b64url(crypto.getRandomValues(new Uint8Array(43)));
const digest = await crypto.subtle.digest(
  "SHA-256", new TextEncoder().encode(verifier));
const challenge = b64url(digest);

sessionStorage.setItem("pkce_verifier", verifier); // survives the redirect
```

Python 3.6 and later

```
import base64, hashlib, secrets

def b64url(b): return base64.urlsafe_b64encode(b).rstrip(b"=").decode()

verifier = b64url(secrets.token_bytes(43))
challenge = b64url(hashlib.sha256(verifier.encode("ascii")).digest())
```

Java 8 and later

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

Base64.Encoder b64url = Base64.getUrlEncoder().withoutPadding();

byte[] random = new byte[43];
new SecureRandom().nextBytes(random);
String verifier = b64url.encodeToString(random);

byte[] digest = MessageDigest.getInstance("SHA-256")
    .digest(verifier.getBytes(StandardCharsets.US_ASCII));
String challenge = b64url.encodeToString(digest);
```

Kotlin — Android

The same JVM primitives. Note `NO_WRAP`: without it Android's encoder inserts line breaks, which will silently corrupt the value.

```
import android.util.Base64
import java.security.MessageDigest
import java.security.SecureRandom

val flags = Base64.URL_SAFE or Base64.NO_PADDING or Base64.NO_WRAP

val random = ByteArray(43).also { SecureRandom().nextBytes(it) }
val verifier = Base64.encodeToString(random, flags)

val digest = MessageDigest.getInstance("SHA-256")
    .digest(verifier.toByteArray(Charsets.US_ASCII))
val challenge = Base64.encodeToString(digest, flags)
```

Swift — iOS and macOS

```
import CryptoKit

func base64url(_ data: Data) -> String {
    data.base64EncodedString()
        .replacingOccurrences(of: "+", with: "-")
        .replacingOccurrences(of: "/", with: "_")
        .replacingOccurrences(of: "=", with: "")
}

var bytes = [UInt8](repeating: 0, count: 43)
_ = SecRandomCopyBytes(kSecRandomDefault, bytes.count, &bytes)

let verifier = base64url(Data(bytes))
let challenge = base64url(Data(SHA256.hash(data: Data(verifier.utf8))))
```

C# — .NET

```
using System.Security.Cryptography;
using System.Text;

static string B64Url(byte[] b) => Convert.ToBase64String(b)
    .Replace('+', '-') .Replace('/', '_') .TrimEnd('=');

var verifier = B64Url(RandomNumberGenerator.GetBytes(43));
var challenge = B64Url(SHA256.HashData(Encoding.ASCII.GetBytes(verifier)));
```

These were run, not typed from memory. The shell, Node, browser, Python, Java and Swift examples were executed against the fixed verifier from Chapter 7 and each reproduced `qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es` exactly. The Kotlin example uses the same JVM primitives as the Java one, and the C# example the .NET equivalents. Whatever language you are in, that vector is how you check your own implementation.

Build the authorization request

Where the challenge goes, what every other parameter is for, and the two values you must keep hold of while the browser is away.

```
// the URL you open in the system browser (line-broken for reading only)
https://login.example.com/authorize
  ?response_type=code
  &client_id=printing-app
  &redirect_uri=com.printingapp%3A%2F%2Fcallback
  &scope=photos.read
  &state=af0ifjsldkj
  &code_challenge=qjrzSW9gMiUgpUvqgEPE4_-8swvyCtf0Vvg55o5S_es
  &code_challenge_method=S256
```

PARAMETER	WHAT IT IS FOR
response_type=code	Asks for the authorization code flow. In OAuth 2.1 this is effectively the only value you should ever send.
client_id	Which application is asking. Public, not secret.
redirect_uri	Where to send the browser afterwards. Must exactly match one the server has pre-registered, character for character. URL-encode it.
scope	What you are asking permission for. Ask for the least you need.
state	Random, per-request. Protects against a different attack; Chapter 16.
code_challenge	The fingerprint. 43 characters for S256.
code_challenge_method	Always S256 .

Two things to keep, and where

While the browser is away, the app must remember the **verifier** and the **state**; losing either means the login cannot be completed. A mobile app keeps them in memory, a single-page app in `sessionStorage`, and a server-side web app in the user's session.

Not `localStorage`, and not a cookie you can read from JavaScript. `localStorage` persists indefinitely and is shared across every tab on the origin, so a verifier left there outlives the login it belonged to. Use `sessionStorage`, and delete the value the moment the token request returns.

Opening the browser

Do not open this URL inside an embedded web view you control. A web view is a window the app can read — including the password the user types into it — which gives back exactly the property OAuth existed to remove. Use the platform's **IN-APP BROWSER TAB** API instead: `ASWebAuthenticationSession` on iOS, `Custom Tabs` on Android. They render the real browser, with its real address bar and its real cookie jar.

Exchange the code

One POST. If you get this request right, PKCE is done.

curl — for trying it by hand

```
curl -s -X POST https://login.example.com/token \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=authorization_code' \
  -d "code=$CODE" \
  -d 'redirect_uri=com.printingapp://callback' \
  -d 'client_id=printing-app' \
  -d "code_verifier=$VERIFIER"
```

JavaScript — and, field for field, any other language

```
const body = new URLSearchParams({
  grant_type: "authorization_code",
  code, // from the redirect
  redirect_uri: "https://app.example.com/callback",
  client_id: "printing-app",
  code_verifier: sessionStorage.getItem("pkce_verifier"),
});

const res = await fetch("https://login.example.com/token", {
  method: "POST",
  headers: { "Content-Type": "application/x-www-form-urlencoded" },
  body,
});

sessionStorage.removeItem("pkce_verifier"); // used once, then gone
const tokens = await res.json();
```

Four things to get right. Send it as `application/x-www-form-urlencoded`, not JSON — the token endpoint takes a form. Send `redirect_uri` again even though it looks redundant; the server compares it with the one from the authorization request. Send the verifier as the *text* you generated, not a re-encoding of it. And do this over HTTPS only: this request carries the one secret in the whole flow.

Break it on purpose

The fastest way to trust a security control is to watch it refuse. Four deliberate failures, and the exact response each one produces.

Run these against any conformant server, or the simulator in the next chapter. Start from a working flow, then change one thing.

1 — The wrong verifier

Complete the flow normally, but send a verifier you made up.

```
-d "code_verifier=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"

HTTP/1.1 400 Bad Request
{ "error": "invalid_grant",
  "error_description": "PKCE verification failed" }
```

This is the attack from Chapter 3, failing. The attacker is in exactly this position: a valid code, and no verifier.

2 — No verifier at all

Omit `code_verifier` entirely. A server that lets this through has the bug from Chapter 10 — an attacker would simply leave the field out.

```
HTTP/1.1 400 Bad Request
{ "error": "invalid_grant",
  "error_description": "code_verifier required" }
```

3 — The code used twice

Exchange a code successfully, then exchange the same code again with the same, correct verifier.

```
HTTP/1.1 400 Bad Request
{ "error": "invalid_grant",
  "error_description": "authorization code already used" }
```

A good server also revokes the tokens it issued the first time: two parties redeeming one code means one is a thief, and the server cannot tell which.

4 — The challenge that is really the verifier

Send `code_challenge_method=S256` but put the raw verifier in `code_challenge`. The commonest beginner bug, and it fails one step later than you expect: the authorization request succeeds, the token request does not.

```
HTTP/1.1 400 Bad Request
{ "error": "invalid_grant",
  "error_description": "PKCE verification failed" }
```

Reading the error you actually got

Every PKCE failure comes back as `invalid_grant`: the server will not help an attacker narrow things down. That makes `error_description` your only clue, and some servers offer none. This is the decision tree to run in your head.

WHAT YOU OBSERVE	ALMOST ALWAYS
Fails every time, from the first attempt	A derivation bug. Check the challenge is 43 characters (Chapter 8) and compare against the fixed vector.
Works in curl, fails in the app	The app is hashing something different from what it sends — usually a trailing newline, or bytes where the text was meant.
Works on the first login, fails on later ones	The verifier is being reused or cached. Generate a new one per attempt.
Works for one user, fails when two log in at once	The verifier is stored somewhere shared — a module-level variable, a singleton, a global. It belongs to the attempt, not the process.
Intermittent, especially on mobile	The app was suspended during the browser step and lost the verifier from memory. Persist it for the duration of the flow.
<code>invalid_request</code> , not <code>invalid_grant</code>	Not PKCE. Usually a <code>redirect_uri</code> mismatch or a missing parameter.
Everything is correct and it still fails	Log the challenge your app sent and the challenge derived from the verifier it later sent, and compare them character by character. They differ; the question is only where.

A debugging trick worth knowing. Temporarily hard-code the verifier and challenge from Chapter 7 in your client. If the flow then succeeds, your generation code is the problem. If it still fails, your *transmission* is — something between your variable and the wire is altering the value.

Run it against the simulator

This book ships next to a working OAuth 2.1 authorization server, API and client, built so that you can turn the safety rules off and watch what happens.

From the repository root:

LAB 1

Watch a correct flow

```
make up && make open
```

Starts the authorization server, the API and the client, and opens the client in your browser. Run a login. Every screen shows the parameters actually in play, including the verifier and the challenge derived from it.

expect: a 43-character challenge in the authorization request, and the verifier appearing only in the token request

LAB 2

Steal a code, and fail to use it

```
make demo-attack
```

Runs Chapter 3 and Chapter 9 on the command line: a code is intercepted and redeemed by a second party that never had the verifier.

expect: invalid_grant, and the real client completing normally afterwards

LAB 3

Turn PKCE off and try again

```
make policy
```

Opens the page where the server's OAuth 2.1 rules can be disabled one at a time. Switch PKCE enforcement off, then run the same attack.

expect: the stolen code now works, and the attacker holds a token

LAB 4

Replay a code

```
make demo-replay
```

Exchanges one code twice, with the correct verifier both times.

expect: the second exchange refused, and the first token revoked

No Docker? `make local` runs the same three services on plain Node. `make test` runs every scenario as a smoke test, and `make help` lists the rest.

PKCE, state and nonce

Three random values in one flow, doing three unrelated jobs. Confusing them is the most common conceptual mistake in OAuth, and it leads people to skip one.

PARAMETER	ANSWERS THE QUESTION	PROTECTS AGAINST
code_verifier code_challenge	Is the app redeeming this code the app that requested it?	Authorization code interception. Checked by the authorization server , at the token endpoint.
state	Did this response come back to a request I actually made?	Cross-site request forgery on the redirect — an attacker tricking your browser into completing <i>their</i> login, so your app ends up holding a token for the attacker's account. Checked by the client .
nonce	Was this ID token minted for this particular login?	ID token replay. An OpenID Connect concept, not an OAuth one. Checked by the client , inside the ID token.

Why you need more than one

The tempting conclusion is that PKCE, being the strongest, makes the others redundant. It does not, because they defend different directions.

- **PKCE protects the server from issuing a token to the wrong app.** It says nothing about whether the *user* at the browser is the user who started the flow.
- **state protects the user from being logged into someone else's account.** An attacker starts a login with their own credentials, captures the redirect containing *their* code, and feeds that URL to your browser. With PKCE alone, your app holds a valid verifier, redeems the attacker's code successfully, and now stores your photos in the attacker's account. `state` is what makes your app notice that it never started this flow.
- **nonce protects the identity claim.** PKCE secures the code; it does not bind an ID token to a login.

Both, always. The RFCs and the current security best-practice document are explicit: PKCE does not replace state. Generate both, from the same secure random source, per attempt. If you are doing OpenID Connect and asking for an ID token, generate a nonce as well.

Do not reuse one value for two jobs

It is occasionally suggested that `state` can be derived from the verifier, or that the challenge can serve as the state. Do not. `state` comes back through the browser in clear text, so anything derived from it is public — and the verifier's entire value is that it is not.

The mistakes beginners make

Ten of them, in roughly the order you are likely to meet them. Most cost an afternoon; two of them cost the security property entirely.

1 Hashing the wrong thing

Hashing raw random bytes instead of the verifier *text*. The authorization request then carries a challenge that has nothing to do with the string you later send. **SYMPTOM** fails every time, from the very first attempt.

2 Standard base64 instead of base64url

`+`, `/` and `=` in a challenge. **SYMPTOM** works locally, fails behind a proxy, or fails only for roughly one attempt in four — whenever the hash happens to contain one of those characters.

3 The trailing newline

`echo` instead of `printf %s` in a shell script; a text editor adding a final newline to a fixture file. **SYMPTOM** consistently wrong challenge with obviously correct code.

4 Reusing the verifier

Generating it once at start-up rather than once per login. **SECURITY** a leaked verifier then compromises every future login, not one. **SYMPTOM** often none — it keeps working, which is what makes it dangerous.

5 Storing the verifier somewhere shared

A module-level variable, a singleton, a global. Two concurrent logins overwrite each other. **SYMPTOM** works in development, fails under load, fails when the user opens two tabs.

6 Losing the verifier across the redirect

Keeping it only in memory in a single-page app, so a full page load wipes it; or on mobile, the app being suspended while the browser is in front. **SYMPTOM** intermittent, and worse on slow networks and older phones.

7 Using plain because it worked first

It is easier to get right, so it is what people reach for when debugging — and then it stays. **SECURITY** restores the original attack in full (Chapter 10). **SYMPTOM** none whatsoever.

8 Sending the verifier in the authorization request

Adding `code_verifier` to the URL “so the server has it”. **SECURITY** publishes the secret in the one channel it must never travel in. **SYMPTOM** the flow works perfectly, which is precisely the problem.

9 An ordinary random number generator

`Math.random()`, `random.random()`, `new Random()`. **SECURITY** a predictable verifier is not a secret. **SYMPTOM** none. This one is only ever found by reading the code.

10 Assuming the server checks

Sending a challenge and trusting that it is enforced. Some servers accept the parameter and never verify it, and some accept a token request with the verifier omitted. **SYMPTOM** Chapter 14, tests 1 and 2, pass when they should fail.

Test for number 10 before you ship. Send a deliberately wrong verifier, and then no verifier at all. If either is accepted, you do not have PKCE — you have a parameter. This is a five-minute test and it is the only one that checks the property you actually care about.

The two that are not bugs

Two things look like mistakes and are not, so that you do not go looking for them:

- **The challenge appears in logs and browser history.** That is expected and safe. It is a hash, and Chapter 8 explains why publishing it costs nothing.
- **The verifier is longer than the challenge.** Also expected. The challenge is a 43-character hash; the verifier is whatever length you generated, commonly 58.

What PKCE does not do

Knowing a control's limits is part of using it. PKCE is narrow on purpose, and several things people expect of it belong to someone else.

IT DOES NOT...	WHICH MEANS
Authenticate the user	PKCE says nothing about who is at the keyboard. That is the authorization server's login, and if your app needs to know <i>who</i> the user is, that is OpenID Connect and an ID token.
Authenticate the client	It proves the redeemer started the flow, not that it is the genuine app. A malicious app can run its own complete PKCE flow perfectly well — with its own verifier, and the user's explicit consent on screen.
Protect the access token	Once issued, the token is a bearer credential: whoever holds it can use it. PKCE protects the <i>exchange</i> , not what comes out of it. Binding a token to a client key is a separate mechanism (DPoP, or mTLS).
Stop a malicious app being installed	Scheme hijacking is still possible; PKCE only makes the stolen code worthless. Preventing the interception itself needs verified redirects — App Links on Android, Universal Links on iOS — which are worth adopting as well.
Replace <code>state</code>	Different attack, different direction, different checker. Chapter 16.
Help if the server does not enforce it	It is a two-party protocol. A client doing PKCE perfectly against a server that ignores it has achieved nothing at all.

What to add alongside it

- **Exact redirect URI matching** on the server — no wildcards, no prefix matching. A sloppy match is an open redirector by another name.
- **Verified redirects** on mobile: App Links and Universal Links prove domain ownership, so the collision in Chapter 3 cannot happen in the first place.
- **Short code lifetimes and one-time use**, with token revocation on reuse.
- **Refresh token rotation**, so a stolen refresh token is detected on second use.
- **A real browser**, never an embedded web view (Chapter 12).

Each of those has a chapter in the companion volume; this book stops at the boundary of PKCE on purpose.

The checklist

What to verify before you ship. The client half takes ten minutes; the server half is the one people skip.

In the client

- ☐ The verifier comes from a **cryptographically secure** random source.
- ☐ A **new** verifier for every authorization attempt — never cached, never reused.
- ☐ Length between 43 and 128 **characters**, from the unreserved set.
- ☐ `code_challenge_method=S256`. Never `plain`.
- ☐ The challenge is **43 characters** and contains no `+`, `/` or `=`.
- ☐ The verifier is stored per-attempt, survives the redirect, and is **deleted** after the token request.
- ☐ The verifier appears in the **token request only** — never in a URL.
- ☐ `state` is also generated, also per attempt, and **checked** on return.
- ☐ The authorization URL opens in a **real browser**, not an embedded web view.
- ☐ The token request goes over **HTTPS**, as a form body.

In the server

- ☐ The challenge and method are stored **with the code**, and both are checked at the token endpoint.
- ☐ A token request with **no** `code_verifier` is **rejected** when a challenge was stored.
- ☐ `plain` is refused, or at minimum cannot be downgraded to.
- ☐ Codes are single-use, short-lived, and reuse **revokes** the tokens already issued.
- ☐ `redirect_uri` is matched **exactly**, with no wildcards.
- ☐ The comparison is constant-time, and failures return a terse `invalid_grant`.
- ☐ `code_challenge_methods_supported` is published in discovery metadata.

The one test that matters. Send a wrong verifier, then none at all. Both must fail. If either succeeds, everything above is decoration.

Quick reference

One page to keep open while you work.

Generate

```
verifier = base64url(random(43 bytes))
           // 58 chars, unreserved set

challenge =
base64url(sha256(ascii(verifier)))
           // always 43 chars
```

Authorization request

```
GET /authorize
    ?response_type=code
    &client_id=...
    &redirect_uri=...
    &scope=...
    &state=<random, per attempt>
    &code_challenge=<43 chars>
    &code_challenge_method=S256
```

Token request

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=...
&redirect_uri=...
&client_id=...
&code_verifier=<the secret>
```

Test vector

```
verifier
M25iVXpKU3puUjFaYWg3T1NDTDQ0t
cW1ROUY5YXlwa1Noc0hhakxifmZHag

challenge
qjrzSW9gMiUgpUvqgEPE4_-8swvy
Ctf0Vvg55o5S_es
```

Line-broken to fit; both are single strings with no whitespace.

Challenge length says what went wrong

43	correct
44	padding ☐ left on
58	you sent the verifier
64	hex-encoded before base64
86	SHA-512, not SHA-256

Shell one-liner

```
V=$(openssl rand -base64 43 \
| tr '+/' '_-' | tr -d '=\\n')

C=$(printf %s "$V" \
| openssl dgst -binary -sha256 \
| openssl base64 \
| tr '+/' '_-' | tr -d '=\\n')
```

Every failure

```
{ "error": "invalid_grant" }
```

Deliberately uninformative. Chapter 14 is the decision tree.

Glossary

Every term this book uses, in the sense this book uses it.

Access token

The credential the app finally gets. Presented to the API to prove it may act. Short-lived, narrow, and a bearer credential — whoever holds it can use it.

Authorization code

A short-lived, single-use string returned through the browser, exchanged at the token endpoint for tokens. Worth stealing; that is what PKCE addresses.

Authorization server

The party that authenticates the user, asks consent, issues codes and tokens, and verifies the PKCE challenge.

base64url

base64 with `-` and `_` in place of `+` and `/`, and padding removed, so the result is safe in a URL.

Bearer token

A token that requires no further proof of identity to use. Its holder is its owner, which is why tokens must not travel through the browser.

Client

The application asking for access. Not the user.

code_challenge

The public fingerprint of the verifier, sent in the authorization request. 43 characters under S256.

code_verifier

The per-login secret, 43–128 characters, sent only in the token request.

Confidential client

A client that can keep a secret, because it runs on a machine the developer controls.

Front channel / back channel

The front channel goes through the user's browser and is visible; the back channel is a direct call between app and server and is not.

ID token

An OpenID Connect artifact describing who logged in. Not an access token, and not something PKCE protects.

Implicit flow

An older flow that returned the access token directly in the redirect. Removed by OAuth 2.1.

invalid_grant

The error returned for every PKCE failure, and for several other things. Deliberately uninformative.

nonce

A random value binding an ID token to one login. OpenID Connect, not OAuth, and not a substitute for anything else.

PKCE

Proof Key for Code Exchange, RFC 7636, said “pixy”. Proves the party redeeming a code is the party that requested it.

plain

The PKCE method where the challenge is the verifier, unhashed. Permitted by the RFC and effectively broken; use S256.

Public client

A client that cannot keep a secret: mobile, desktop, single-page, CLI. Everything shipped to a user is public.

Redirect URI

Where the authorization server sends the browser afterwards. Must be pre-registered and matched exactly.

Refresh token

A longer-lived credential for getting new access tokens without another login. Should rotate on every use.

S256

The PKCE method where the challenge is the base64url of the SHA-256 of the verifier. The one to use.

Scope

What the app is asking permission to do. Ask for the least that works.

state

A random per-request value the client sends and checks on return, defending against a forged redirect. Not replaced by PKCE.

Questions people ask

The ones that come up every time, and where to read next.

My client has a client secret. Do I still need PKCE?

Yes. OAuth 2.1 requires PKCE for *all* clients using the authorization code flow, confidential ones included. The secret proves which application is calling; PKCE proves the call finishes the flow it started. They are different guarantees, and the second one also protects against a leaked code inside your own infrastructure.

Is PKCE only for mobile apps?

No. It was written with mobile in mind and is now the baseline for every client type. Single-page apps need it most acutely, since they are public clients whose redirect is a plain browser URL.

Can I reuse one verifier for a session?

No. One per authorization attempt. A reused verifier turns a single leak into a permanent one, and if two flows run concurrently they will overwrite each other.

The challenge shows up in my server logs. Is that a problem?

No. It is a hash, and it is useless without the verifier. That is exactly why PKCE hashes it. The *verifier* in a log would be a problem — check that your token endpoint does not log request bodies.

What if the authorization server does not support PKCE?

Then you do not have this protection, and no amount of client-side correctness supplies it. Check `code_challenge_methods_supported` in the discovery document. If it is absent, raise it with whoever runs the server; if it is your server, Chapter 19 lists what it must do.

Does PKCE mean I can put a token in a single-page app safely?

It means the code exchange is safe. What happens to the token afterwards is a separate question with a separate answer — token storage, XSS, and rotation are all covered in the companion volume.

Where to read next

RFC 7636	PKCE itself. Short, readable, and the normative source for everything in this book.
RFC 9700	Best Current Practice for OAuth 2.0 Security. Why PKCE is now required everywhere, plus the attacks it does not cover.
OAuth 2.1 draft	The consolidation: PKCE mandatory, implicit flow removed, exact redirect matching.
RFC 8252	OAuth for Native Apps. Custom schemes, verified redirects, and why not to use a web view.
The companion volume	<i>The Ultimate Developer's Guide to OAuth 2.1 and PKCE</i> — tokens, scopes, refresh rotation, OpenID Connect, and the simulator's source in full.

If you remember one thing. PKCE does not stop the authorization code being stolen. It makes a stolen code useless, by requiring a secret that never traveled through the channel the thief was listening to.